

Приватний заклад вищої освіти
Одеський технологічний університет «ШАГ»
Кафедра інформаційних технологій та фундаментальної підготовки

випускна кваліфікаційна робота бакалавра

**Інформаційна платформа розповсюдження мультимедійного контенту за
моделлю підписки (на прикладі сервісу Bazinga)**

на здобуття бакалаврського рівня вищої освіти
зі спеціальності «F3 Комп'ютерні науки»

Виконавці проекту:

№	П.І.Б.	Ролі	Група
1	Маймескул Єгор Олегович	Full-Stack	КНП-221
2	Руденко Олена Костянтинівна	Full-Stack	КНП-221

Автор звіту: Маймескул Єгор Олегович

Керівник	Доцент кафедри ІТ та ФП, к.т.н., [Прізвище І. П. керівника]
----------	---

Одеса – 2025

АНОТАЦІЯ

УДК 004.9

Маймескул Є. О. Інформаційна платформа розповсюдження мультимедійного контенту за моделлю підписки (на прикладі сервісу Bazinga). Підсистема ідентифікації, керування обліковими записами та платіжно-підписного обслуговування. — Пояснювальна записка до випускної кваліфікаційної роботи на здобуття бакалаврського рівня вищої освіти зі спеціальності «Ф3 Комп'ютерні науки» (0613 Software and applications development and analysis). — Одеса: ОТУШ, 2025.

Об'єктом дослідження є процеси ідентифікації, автентифікації та платіжно-підписного обслуговування користувача в межах розподіленої інформаційної системи розповсюдження мультимедійного контенту за моделлю підписки.

Метою роботи є підвищення інформаційної безпеки та відмовостійкості підсистеми керування обліковими записами шляхом синтезу структурних і функціональних моделей процесів реєстрації, багатопрофільного доступу та платіжного обслуговування в умовах неповної визначеності щодо доступності зовнішніх сервісів.

Методи дослідження утворені суперпозицією методів системного аналізу, об'єктно-орієнтованого моделювання, порівняльного аналізу проєктних рішень та експериментального випробування. Інструментальне забезпечення: платформа .NET 9 (ASP.NET Core 9, Entity Framework Core 9) над системою керування базами даних MySQL 8; клієнтський застосунок на React 18 і TypeScript 5 зі складанням засобами Vite 5; контейнеризація — Docker.

Результати та їх новизна. Спроектовано та впроваджено підсистему облікових записів, у якій реалізовано безпарольну реєстрацію за одноразовим посиланням, багатопрофільну модель доступу (до п'яти профілів на обліковий запис), захист окремого профілю чотиризначним PIN-кодом, двофакторну автентифікацію за стандартом TOTP (RFC 6238), відновлення доступу через одноразове посилання та відмовостійку інтеграцію з платіжним шлюзом. Новизна полягає у поєднанні зазначених засобів захисту в єдиній підсистемі з градаційним зниженням функціональності (graceful degradation) при недоступності зовнішніх комунікаційних сервісів, що відрізняє запропоноване рішення від наявних аналогів і підвищує його відмовостійкість.

Основні характеристики. Реалізовано 13 контролерів і 8 інжектів сервісів серверної частини; одноразові токени зберігаються виключно у вигляді SHA-256-згорток; паролі — у вигляді адаптивних BCrypt-згорток; сесія підтверджується підписаним маркером JWT (HMAC SHA-256).

Сфера застосування — інформаційні системи розповсюдження цифрового контенту за моделлю підписки, системи е-комерції та е-послуг. *Значущість роботи* визначається відповідністю напряму державної політики щодо розвитку цифрової економіки та інформаційного суспільства. *Взаємозв'язок з іншими роботами* — підсистема є складовою командного проекту Bazinga; друга підсистема (каталог контенту, агрегатор метаданих і потокове відео) реалізована другим виконавцем.

Пояснювальна записка складається зі вступу, технічного завдання, трьох розділів, висновків, переліку використаних джерел та додатка з прикладами програмного коду. При складанні звіту використано 20 джерел інформації.

ЗМІСТ

- ЗМІСТ
 - ВСТУП
 - ТЕХНІЧНЕ ЗАВДАННЯ ДО ПРОЄКТУ
 - РОЗДІЛ 1. ЗАГАЛЬНА ХАРАКТЕРИСТИКА СИСТЕМИ ТА ПОРІВНЯЛЬНИЙ АНАЛІЗ
 - РОЗДІЛ 2. ПРОЄКТУВАННЯ ТА РЕАЛІЗАЦІЯ ПІДСИСТЕМИ
 - РОЗДІЛ 3. ВИПРОБУВАННЯ ТА ОЦІНКА РЕЗУЛЬТАТІВ
 - ВИСНОВКИ
 - ПЕРЕЛІК ВИКОРИСТАНИХ ДЖЕРЕЛ
 - ДОДАТОК А. ПРИКЛАДИ ПРОГРАМНОГО КОДУ
-

ВСТУП

Оцінка сучасного стану об'єкта дослідження. Останнє десятиліття характеризується істотним зрушенням споживання мультимедійного контенту до інформаційних систем цифрової дистрибуції, що функціонують за моделлю «контент за підпискою». Подібні системи охоплюють як вузькоспеціалізовані сервіси читання коміксів і манги, так і універсальні системи потокового перегляду відеоконтенту. Спільною ознакою цих систем є винесення на перший план таких функціональних складових, як безпечна реєстрація, прозорий цикл підписки, багатoproфільне використання облікового запису та захист персональних і платіжних даних.

Світові тенденції розв'язання задачі. Провідні системи поступово відмовляються від класичної пари «логін + пароль» на користь безпарольних механізмів автентифікації (passwordless, magic link, одноразові коди), а також впроваджують багаторівневі засоби захисту доступу — двофакторну автентифікацію та PIN-захист окремих профілів. Паралельно діє вимога винесення обробки платіжних даних за межі власної інформаційної системи (мінімізація області застосування стандарту PCI-DSS) шляхом застосування спеціалізованих платіжних шлюзів.

Актуальність роботи. Реалізація захищеної підсистеми керування обліковими записами та платіжного обслуговування є передумовою побудови будь-якої сучасної інформаційної системи розповсюдження контенту за моделлю підписки, що й обумовлює актуальність роботи. Додатковим чинником актуальності є необхідність підвищення інформаційної безпеки персональних і платіжних даних користувача за умов зростання кількості автоматизованих атак на засоби автентифікації.

Умови невизначеності. Задача проектування підсистеми сформульована в умовах неповної визначеності вихідних даних: зовнішні комунікаційні сервіси (платіжний шлюз, поштовий сервер) можуть бути недоступними, обмежувати кількість звернень або відповідати з істотною затримкою. Це не дозволяє вважати задачу детермінованою і потребує дослідницького підходу — порівняння альтернатив, обрання критеріїв та проектування механізмів, що зберігають працездатність системи за відмови окремих її зовнішніх складових.

Наукова новизна. Запропоноване рішення не є простим відтворенням наявних систем. Новизна полягає у синтезі структурної моделі підсистеми облікового запису, яка поєднує безпарольну реєстрацію, багатoproфільність із PIN-захистом, двофакторну автентифікацію за стандартом TOTP та відмовостійкий платіжний потік із градаційним зниженням функціональності за відмови зовнішніх сервісів. Порівняльний аналіз (розділ 1) показує, що жодна з розглянутих систем-аналогів не поєднує усіх зазначених властивостей одночасно.

Мета і задачі. Метою роботи є проектування, реалізація та впровадження захищеної й відмовостійкої підсистеми керування обліковими записами, профілями та платіжно-підписною моделлю інформаційної системи Bazinga. Для досягнення мети поставлено такі задачі:

1. провести огляд інформаційних систем розповсюдження контенту за підпискою, виконати порівняльний аналіз їхніх засобів ідентифікації та захисту доступу, встановити напрями удосконалення;
2. синтезувати структурну інформаційну модель даних облікового запису, профілів та одноразових токенів; забезпечити ідемпотентне розгортання схеми;
3. реалізувати серверну частину підсистеми — комунікаційні інтерфейси (контролери) та інфраструктурні сервіси автентифікації, поштового обслуговування та платіжної інтеграції;
4. впровадити засоби інформаційної безпеки: криптографічне зберігання облікових даних, безпарольну реєстрацію, відновлення доступу, PIN-захист профілів та двофакторну автентифікацію;
5. реалізувати клієнтську частину підсистеми та узгодити стан клієнта і сервера засобами комунікаційного протоколу REST поверх HTTP із підтвердженням маркером JWT;
6. провести випробування підсистеми за різних умов, зокрема за відмови зовнішніх сервісів, та оцінити відповідність технічному завданню.

Сфера застосування та взаємозв'язок з іншими роботами. Результати роботи застосовні в інформаційних системах розповсюдження цифрового контенту, системах е-комерції та е-послуг. Робота є складовою командного проєкту; підсистема каталогу контенту, агрегатора метаданих та потокового відео реалізована другим виконавцем і описана в окремій пояснювальній записці.

ТЕХНІЧНЕ ЗАВДАННЯ ДО ПРОЄКТУ

Загальний опис

Програмне забезпечення реалізує підсистему ідентифікації, керування обліковими записами, профілями та платіжно-підписним обслуговуванням розподіленої інформаційної системи Bazinga, що надає доступ до двох категорій контенту — коміксів/манги та потокового відео — за моделлю місячної підписки.

Архітектурно система поділяється на дві логічні складові. Перша — *Bazinga Comics* — забезпечує перегляд каталогу коміксів і манги та читання випусків онлайн. Друга — *BazingaTV* — забезпечує потоковий перегляд відеоконтенту. Підсистема, що реалізована у межах цієї роботи, є спільною для обох складових та охоплює увесь шлях користувача — від заявки на реєстрацію до обрання активного профілю.

Призначення

Підсистема призначена для виконання таких користувацьких сценаріїв:

- реєстрація нового користувача через електронну пошту (безпарольний сценарій із підтвердженням посиланням);
- обрання тарифного плану підписки та прив'язка платіжного інструменту;
- керування обліковим записом: зміна персональних даних, номера телефону, перегляд відомостей про підписку;
- створення, редагування та видалення профілів у межах облікового запису, захист профілю PIN-кодом;
- увімкнення двофакторної автентифікації та відновлення доступу за одноразовим посиланням;
- обрання активного профілю при кожному новому відвідуванні; безпечне завершення сесії.

Функціональні вимоги до підсистеми

Реєстрація та автентифікація:

- реєстрація починається з введення електронної пошти та миттєвої перевірки її зайнятості;
- одноразове посилання-токен дійсне 15 хвилин і має одноразовий характер;
- завершення реєстрації реалізується як чотирикроковий процес: Welcome → Password → Plan → Card;
- автентифікація постійних користувачів — за електронною поштою і паролем або за одноразовим посиланням (passwordless sign-in);

- передбачено відновлення паролю за одноразовим посиланням та можливість увімкнення двофакторної автентифікації (TOTP).

Профілі:

- один обліковий запис містить до п'яти профілів; перший (кореневий) створюється автоматично;
- профіль зберігає ім'я, аватар, статус «Kids» та (опціонально) PIN-код;
- активний профіль зберігається на час сесії та повторно запитується при кожному новому відвідуванні;
- профіль із встановленим PIN-кодом потребує його введення перед обранням.

Підписка та оплата:

- тарифи: Bazinga Comics, BazingaTV, Bazinga Unlimited та пробний 7-денний Trial;
- введення даних картки — виключно через офіційний компонент платіжного шлюзу (винесення області PCI-DSS);
- для середовища без сконфігурованого шлюзу — резервний режим, що дозволяє завершити демонстраційний сценарій.

Нефункціональні вимоги

- паролі зберігаються виключно у вигляді BCrypt-згорток; одноразові токени — у вигляді SHA-256-згорток;
- маркер JWT підписується алгоритмом HMAC SHA-256;
- усі звернення до платіжного шлюзу обмежено тайм-аутом (12 с) із одним автоматичним повтором;
- оновлення схеми бази даних виконується ідемпотентно, без ручного втручання адміністратора;
- клієнтська частина адаптивна (екрани від 320 px до 4K); підтримуються актуальні версії основних браузерів.

Архітектура та технології

Систему спроектовано за клієнт-серверною моделлю: серверна частина — ASP.NET Core 9 (stateless REST-сервіс); клієнтська — SPA на React 18 + TypeScript, що складається засобами Vite та обслуговується NGINX; сховище даних — MySQL через Pomelo.EntityFrameworkCore.MySql; допоміжні сервіси — SMTP та платіжний шлюз Stripe. Розгортання — Docker і Docker Compose.

РОЗДІЛ 1. ЗАГАЛЬНА ХАРАКТЕРИСТИКА СИСТЕМИ ТА ПОРІВНЯЛЬНИЙ АНАЛІЗ

Інформаційна система Bazinga є розподіленою клієнт-серверною системою, побудованою за архітектурним стилем «односторінковий застосунок (SPA) поверх комунікаційного протоколу REST». Система поєднує дві предметні складові — розповсюдження коміксів/манги (Bazinga Comics) та потокове відео (BazingaTV) — у межах єдиного облікового запису. Цей розділ присвячено загальній характеристиці системи та порівняльному аналізу наявних рішень, за результатами якого обґрунтовано предмет розроблення підсистеми облікових записів.

Серверна складова реалізована на платформі ASP.NET Core 9 із застосуванням об'єктно-реляційного відображення Entity Framework Core поверх MySQL. Комунікаційні інтерфейси побудовано за конвенціями REST: семантика методів GET/POST/PUT/DELETE та коди стану HTTP узгоджені з RFC 7231; підтвердження суб'єкта виконується маркером JWT (RFC 7519), підписаним алгоритмом HMAC SHA-256. Клієнтська складова реалізована мовою TypeScript на основі бібліотеки React зі складанням засобами Vite; керування серверним станом виконує бібліотека TanStack Query, маршрутизацію — React Router.

Функціонально систему поділено на дві підсистеми відповідно до розподілу обов'язків у команді: (1) підсистема облікових записів, профілів та підписок (предмет цієї роботи) і (2) підсистема каталогу контенту, агрегатора метаданих та потокового відео (реалізована другим виконавцем). Взаємодія підсистем відбувається через спільний контекст автентифікації на боці клієнта та атрибут авторизації на боці сервера, а також через спільну структурну модель даних, у якій сутність «користувач» є центральним вузлом.

Порівняльний аналіз наявних рішень. Для обґрунтування предмета розроблення проведено порівняння інформаційних систем-аналогів за ознаками, що стосуються ідентифікації користувача та захисту доступу. Результати наведено у таблиці 1.1.

Таблиця 1.1 — Порівняльний аналіз засобів ідентифікації та захисту доступу

Ознака / Система	Netflix	Crunchyroll	ComiXology	Bazinga
Єдиний обліковий запис для відео та коміксів/манги	–	–	–	+
Безпарольна реєстрація за магічним посиланням	+	–	–	+
Багатопрофільність (до 5 профілів)	+	–	–	+
PIN-захист окремого профілю	+	–	–	+
Двофакторна автентифікація (TOTP)	–	–	–	+

Ознака / Система	Netflix	Crunchyroll	ComiXology	Bazinga
Підтвердження номера телефону для майбутньої 2FA	+	–	–	+
Відмовостійкий платіжний потік із резервним режимом	–	–	–	+

Як видно з таблиці 1.1, у жодному рядку немає одночасної наявності ознаки в усіх системах, що підтверджує диференційність обраних критеріїв. Найбільшу кількість позитивних ознак серед систем-аналогів має Netflix; саме його взято за взірцеву систему щодо моделі багатoproфільного доступу. Водночас жодна з наявних систем не поєднує двофакторної автентифікації, PIN-захисту профілю та відмовостійкого платіжного потоку з єдиним обліковим записом для різноманітного контенту — саме усунення цих «мінусів» становить предмет удосконалення у даній роботі.

Міні-висновок за розділом. За результатами порівняльного аналізу встановлено, що саме розробляється — підсистема облікових записів із підвищеною інформаційною безпекою та відмовостійкістю; чому не застосовано готове рішення — наявні системи не поєднують потрібного набору властивостей і не призначені для єдиного обслуговування різноманітного контенту; для чого це потрібно — побудова такої підсистеми є передумовою функціонування інформаційної системи розповсюдження контенту за підпискою, розвиток яких визначено державним пріоритетом. Це обґрунтовує предмет і задачі подальшого проектування.

РОЗДІЛ 2. ПРОЄКТУВАННЯ ТА РЕАЛІЗАЦІЯ ПІДСИСТЕМИ

Розділ присвячено проєктним рішенням та їх реалізації — усьому, що передувало першому успішному запуску підсистеми. Викладено обґрунтування вибору інструментальних засобів, структурну модель даних, комунікаційні інтерфейси, інфраструктурні сервіси, засоби інформаційної безпеки та клієнтську частину. Фрагменти програмного коду, що ілюструють ключові рішення, винесено у додаток А із посиланнями за текстом.

2.1. Обґрунтування вибору інструментальних засобів

Вибір інструментальних засобів виконано методом порівняльного аналізу. Для серверної платформи розглянуто ASP.NET Core 9, Node.js (Express) та Spring Boot. Обрано ASP.NET Core 9 з огляду на статичну типізацію мови C#, високі показники продуктивності у незалежних порівняннях (TechEmpower) та зрілість екосистеми (вбудована підтримка JWT, EF Core, OpenAPI). Для об'єктно-реляційного відображення обрано Entity Framework Core 9 з провайдером Pomelo для MySQL.

Для криптографічного перетворення паролів розглянуто алгоритми PBKDF2, BCrypt та Argon2. Обрано BCrypt (бібліотека BCrypt.Net-Next) як адаптивний алгоритм із налаштовуваним фактором роботи, рекомендований OWASP та реалізований у зрілій, широко застосовуваній бібліотеці. Для підтвердження сесії обрано маркер JWT (RFC 7519), оскільки він не потребує серверного стану і природно узгоджується зі stateless-архітектурою. Для поштового обслуговування обрано бібліотеку MailKit, для платіжної інтеграції — офіційну бібліотеку Stripe.NET. Для двофакторної автентифікації обрано алгоритм TOTP (RFC 6238), сумісний із поширеними застосунками-автентифікаторами, реалізований без додаткових залежностей власними засобами (див. п. 2.4).

Для клієнтської частини обрано React 18 + TypeScript 5 зі складанням засобами Vite. Статична типізація знижує кількість помилок на етапі компіляції; компонентна модель забезпечує повторне використання елементів інтерфейсу. Бібліотека shadcn/ui поверх примітивів Radix UI застосована як набір типових інтерфейсних елементів.

2.2. Структурна модель даних

Синтезовано структурну інформаційну модель, центральним вузлом якої є сутність *User* (обліковий запис). До неї каскадно прив'язані сутність *Profile* (профіль) та одноразові токени. Модель нормалізовано до третьої нормальної форми.

Сутність *User* містить унікальні атрибути імені користувача та електронної пошти, BCrypt-згортку паролю, номер телефону, тип і термін підписки, а також атрибути двофакторної автентифікації — прапорець активації та секрет TOTP (що позначений як прихований при серіалізації). Сутність *Profile* містить посилання на власника, ім'я, аватар, прапорці кореневого та дитячого профілю та (опціонально) BCrypt-згортку PIN-

коду. Одноразові токени (реєстрації, входу, відновлення паролю, виклику 2FA) зберігаються виключно у вигляді SHA-256-згорток із атрибутами терміну дії та факту використання — оригінальне значення посилання у сховищі відсутнє.

Ідемпотентне розгортання схеми реалізовано викликами `CREATE TABLE IF NOT EXISTS` та умовним додаванням стовпців, що дозволяє оновлювати наявні екземпляри без ручного втручання та без втрати даних (приклад 1 ілюструє генерацію та згортання токена).

2.3. Комунікаційні інтерфейси (контролери)

Комунікаційний інтерфейс підсистеми реалізовано контролером `AuthController`, що надає ендпоінти безпарольної реєстрації (перевірка пошти, ініціювання, верифікація токена, фіналізація), безпарольного входу, відновлення паролю та двофакторної автентифікації, а також контролером `ProfilesController`, що реалізує повний цикл операцій над профілями та керування PIN-кодом. Усі захищені ендпоінти позначено атрибутом авторизації; поточний суб'єкт визначається з клеймів маркера JWT.

Ініціювання безпарольної реєстрації (приклад 2) генерує одноразовий токен, інвалідує попередні активні токени для цієї адреси та надсилає комунікаційним каналом електронної пошти повідомлення з посиланням. Фіналізація (приклад 3) перевіряє стан токена, створює обліковий запис із обраним тарифом, атомарно формує кореневий профіль та повертає підписаний маркер сесії.

2.4. Засоби інформаційної безпеки

Засоби інформаційної безпеки утворюють багаторівневу модель захисту доступу.

Криптографічне зберігання облікових даних. Паролі та PIN-коди перетворюються адаптивним алгоритмом BCrypt (приклад 5); відновлення оригіналу зі згортки є обчислювально складним. Маркер сесії формується та підписується алгоритмом HMAC SHA-256 (приклад 4).

Безпарольні сценарії. Реєстрація, вхід та відновлення паролю реалізовані за одноразовими посиланнями; у сховищі зберігається лише SHA-256-згортка токена з обмеженим терміном дії та ознакою одноразовості (приклад 1). Це усуває ризик експлуатації перехопленого або викраденого з бази посилання.

Двофакторна автентифікація. Реалізовано алгоритм TOTP (RFC 6238) власними засобами без зовнішніх залежностей: генерацію секрету у кодуванні Base32, формування `otpauth-URI` для застосунку-автентифікатора та перевірку коду з допуском відхилення годинника у межах одного інтервалу (приклад 6). За увімкненої 2FA перший фактор (пароль або посилання) не завершує сесію, а повертає одноразовий виклик (challenge), який обмінюється на маркер лише після перевірки коду (приклад 7).

Захист профілю. Окремий профіль може бути захищений чотиризначним PIN-кодом; його згортка перевіряється перед обранням профілю, при цьому кореневий профіль може скидати PIN дочірніх профілів (батьківський контроль) — приклад 8.

2.5. Відмовостійка платіжна інтеграція

Інтеграцію з платіжним шлюзом побудовано за принципом мінімізації області застосування стандарту PCI-DSS: чутливі дані картки не потрапляють на сервер системи, а збираються офіційним компонентом шлюзу. Сервер створює об'єкти `Customer` та `SetupIntent` і повертає клієнту одноразовий секрет (приклад 9). Звернення до шлюзу обмежено тайм-аутом 12 с з одним автоматичним повтором; за відмови шлюзу система переходить у резервний режим, що дозволяє завершити сценарій, — це реалізація градаційного зниження функціональності в умовах невизначеності щодо доступності зовнішнього сервісу.

2.6. Клієнтська частина та узгодження стану

Клієнтську частину реалізовано як сукупність сторінок безпарольної реєстрації, входу, відновлення паролю, селектора та редактора профілю й особистого кабінету. Завершення реєстрації реалізовано як скінченний автомат із чотирма станами.

Стан автентифікації централізовано у контексті `AuthContext`. Застосовано диференційоване зберігання за терміном дії: відомості облікового запису та маркер сесії зберігаються довгостроково, а ідентифікатор активного профілю — лише на час сесії, що реалізує вимогу повторного запиту профілю при кожному новому відвідуванні (приклад 10). Захист маршрутів реалізовано охоронними компонентами `RequireAuth` та `RequireProfile`, які запобігають доступу до контенту без підтвердженої сесії та обраного профілю (приклад 11). Введення PIN-коду та налаштування двофакторної автентифікації реалізовано окремими інтерфейсними компонентами (приклад 12).

РОЗДІЛ 3. ВИПРОБУВАННЯ ТА ОЦІНКА РЕЗУЛЬТАТІВ

Розділ присвячено перевірці підсистеми після першого успішного запуску: відповідності технічному завданню, функціональному та безпековому випробуванню, оцінці продуктивності й відмовостійкості, а також застосованим засобам контролю якості.

3.1. Відповідність технічному завданню

Перевірку відповідності виконано шляхом проходження повних користувацьких сценаріїв. Підтверджено реалізацію всіх функціональних вимог: безпарольної реєстрації; безпарольного й парольного входу; відновлення паролю; багатoproфільності з обмеженням «не більше п'яти профілів» та заборонаю видалення кореневого профілю; PIN-захисту профілю; двофакторної автентифікації; платіжно-підписного циклу з резервним режимом. Сценарій «реєстрація → підтвердження пошти → обрання тарифу → прив'язка картки → обрання профілю → доступ до контенту» виконується безперервно.

3.2. Функціональне випробування

Функціональне випробування проведено для основних сценаріїв у браузерях на базі рушіїв Chromium та Gecko. Перевірено: коректність переходів скінченного автомата реєстрації та збереження введених даних під час навігації між кроками; автоматичне створення рівно одного кореневого профілю (зокрема за умов подвійного виклику ефектів у режимі розробки); повторний запит профілю після перезапуску браузера; коректне відображення стану підписки та блокування доступу після завершення пробного тарифу.

3.3. Безпекове випробування

Безпекове випробування охопило типові вектори. Перевірено, що: повторне використання одноразового посилання відхиляється (стан «використано»); прострочений токен відхиляється (стан «прострочено»); підроблення маркера JWT без знання серверного секрету призводить до відмови в авторизації; введення некоректного PIN-коду чи коду TOTP не надає доступу; дані картки не передаються на сервер системи (підтверджено інспекцією мережесх звернень). Зберігання паролів і токенів виключно у вигляді згорток підтверджено інспекцією таблиць бази даних.

3.4. Оцінка продуктивності та відмовостійкості

Час відповіді ендпоінтів автентифікації за середнього навантаження не перевищує визначеного у технічному завданні порога у 300 мс. Відмовостійкість підтверджено випробуванням у штучних умовах невизначеності: за блокування звернень до платіжного шлюзу клієнт завершує сценарій через резервний режим у межах сумарного тайм-ауту,

без «зависання» інтерфейсу; за відсутності налаштованого поштового сервера сценарій реєстрації відтворюється повністю завдяки резервному режиму журналювання посилення.

3.5. Контроль якості та командна взаємодія

Розроблення велося із застосуванням системи контролю версій Git за моделлю гілок, запитів на злиття (pull request) та взаємного рецензування коду. Якість клієнтського коду підтримувалася статичним аналізатором та засобами форматування; типобезпека перевірялася компілятором TypeScript, складання — засобами Vite. Інтеграція підсистем здійснювалася через узгоджені комунікаційні інтерфейси, що знизило зв'язність складових і спростило супровід.

3.6. Підсумкова оцінка

Функціональність, передбачена технічним завданням, реалізована повністю. Підсистема забезпечує надійне виконання всіх ключових сценаріїв ідентифікації, керування доступом та платіжного обслуговування, зберігаючи працездатність за відмови зовнішніх сервісів. Обрані інструментальні засоби та проєктні рішення підтвердили доцільність; підсистема є придатною до подальшого масштабування завдяки stateless-архітектурі серверної частини.

ВИСНОВКИ

У ході виконання випускної кваліфікаційної роботи спроектовано та впроваджено захищену й відмовостійку підсистему керування обліковими записами, профілями та платіжно-підписною моделлю інформаційної системи Bazinga. Висновки за поставленими у вступі задачами наведено нижче.

- **Задача 1 (огляд і порівняння).** Проведено огляд інформаційних систем розповсюдження контенту за підпискою та порівняльний аналіз за сімома ознаками засобів ідентифікації й захисту доступу (таблиця 1.1). Встановлено, що жодна із систем-аналогів не поєднує єдиного облікового запису для різнорідного контенту з двофакторною автентифікацією, PIN-захистом профілю та відмовостійким платіжним потоком; це визначило предмет удосконалення.
- **Задача 2 (модель даних).** Синтезовано структурну інформаційну модель облікового запису, профілів і одноразових токенів, нормалізовану до ЗНФ; реалізовано ідемпотентне розгортання схеми засобами `CREATE TABLE IF NOT EXISTS` та умовного додавання стовпців, що усуває потребу ручного супроводу схеми.
- **Задача 3 (серверна частина).** Реалізовано комунікаційні інтерфейси (контролери `AuthController`, `ProfilesController`) та інфраструктурні сервіси автентифікації, поштового обслуговування й платіжної інтеграції — загалом у складі системи 13 контролерів і 8 інжектованих сервісів.
- **Задача 4 (інформаційна безпека).** Впроваджено багаторівневу модель захисту: BCrypt-згортки паролів і PIN-кодів, SHA-256-згортки одноразових токенів, підписаний маркер JWT, безпарольні реєстрацію/вхід/відновлення, PIN-захист профілю та двофакторну автентифікацію за стандартом TOTP, реалізовану власними засобами без зовнішніх залежностей.
- **Задача 5 (клієнтська частина).** Реалізовано клієнтську частину з диференційованим зберіганням стану (довгострокове — для облікового запису, сесійне — для активного профілю) та охоронними компонентами маршрутизації; узгодження клієнта і сервера виконано засобами комунікаційного протоколу REST із підтвердженням маркером JWT.
- **Задача 6 (випробування).** Проведено функціональне та безпекове випробування; підтверджено відповідність технічному завданню, час відповіді ендпоінтів автентифікації не перевищує 300 мс, а працездатність зберігається за відмови платіжного та поштового сервісів завдяки градаційному зниженню функціональності.

Наукова новизна роботи полягає у синтезі структурної моделі підсистеми облікового запису, що поєднує перелічені засоби захисту з відмовостійкою поведінкою в умовах неповної визначеності щодо доступності зовнішніх сервісів. Практична цінність визначається застосовністю результатів в інформаційних системах е-комерції та е-послуг.

Перспективними напрямками подальших досліджень є впровадження апаратних ключів автентифікації (WebAuthn/FIDO2), інтеграція з зовнішніми провайдерами ідентифікації та порівняльне дослідження криптографічних примітивів на основі національних стандартів за критеріями надійності й швидкодії.

ПЕРЕЛІК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. ДСТУ 3008:2015. Інформація та документація. Звіти у сфері науки і техніки. Структура та правила оформлювання. Київ: ДП «УкрНДНЦ», 2016.
2. ДСТУ 8302:2015. Інформація та документація. Бібліографічне посилання. Загальні положення та правила складання. Київ: ДП «УкрНДНЦ», 2016.
3. Величко О. М., Гордієнко Т. Б. Інтелектуальні інформаційні системи: структура і застосування: підручник. Херсон, 2022. 728 с. ISBN 978-966-289-552-0.
4. Анісімов А. В., Кулябко П. П. Інформаційні системи та бази даних: навчальний посібник. Київ, 2021. 110 с.
5. M. Jones, J. Bradley, N. Sakimura. JSON Web Token (JWT). RFC 7519. IETF, 2015. URL: <https://datatracker.ietf.org/doc/html/rfc7519>
6. D. M'Raihi, S. Machani, M. Pei, J. Rydell. TOTP: Time-Based One-Time Password Algorithm. RFC 6238. IETF, 2011. URL: <https://datatracker.ietf.org/doc/html/rfc6238>
7. S. Josefsson. The Base16, Base32, and Base64 Data Encodings. RFC 4648. IETF, 2006. URL: <https://datatracker.ietf.org/doc/html/rfc4648>
8. M. Jones, D. Hardt. The OAuth 2.0 Authorization Framework: Bearer Token Usage. RFC 6750. IETF, 2012. URL: <https://datatracker.ietf.org/doc/html/rfc6750>
9. OWASP Authentication Cheat Sheet. OWASP Foundation, 2024. URL: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
10. OWASP Application Security Verification Standard (ASVS) v4.0.3. OWASP Foundation, 2021. URL: <https://owasp.org/www-project-application-security-verification-standard/>
11. PCI Security Standards Council. PCI DSS v4.0. 2022. URL: <https://www.pcisecuritystandards.org/>
12. ASP.NET Core documentation. Microsoft, 2024. URL: <https://learn.microsoft.com/aspnet/core>
13. Entity Framework Core documentation. Microsoft, 2024. URL: <https://learn.microsoft.com/ef/core>
14. React documentation. Meta Open Source, 2024. URL: <https://react.dev/learn>
15. TypeScript documentation. Microsoft, 2024. URL: <https://www.typescriptlang.org/docs/>
16. Vite documentation. 2024. URL: <https://vite.dev/guide/>
17. Stripe API Reference. Stripe, 2024. URL: <https://stripe.com/docs/api>
18. MailKit documentation. 2024. URL: <https://github.com/jstedfast/MailKit>
19. MySQL 8.0 Reference Manual. Oracle, 2024. URL: <https://dev.mysql.com/doc/refman/8.0/en/>

20. Репозиторій проекту Bazinga. URL: <https://github.com/dentuss/Bazinga>

ДОДАТОК А. ПРИКЛАДИ ПРОГРАМНОГО КОДУ

У додатку наведено приклади програмного коду, що ілюструють ключові проєктні рішення підсистеми. Повний вихідний код розміщено у репозиторії проєкту (джерело 20 переліку використаних джерел).

Приклад 1 — Генерація одноразового токена та його незворотне SHA-256-згоргання (`Security/SignupTokens.cs`)

```
public static class SignupTokens
{
    public static string Generate()
    {
        var bytes = RandomNumberGenerator.GetBytes(32);    // 256 біт ентропії
        return Convert.ToBase64String(bytes)
            .Replace('+', '-').Replace('/', '_').TrimEnd('=');
    }

    public static string Hash(string token)
    {
        var bytes = SHA256.HashData(Encoding.UTF8.GetBytes(token));
        return Convert.ToHexString(bytes);                // у БД зберігається лише
згортка
    }
}
```

Приклад 2 — Ініціювання безпарольної реєстрації: інвалідація попередніх токенів та надсилання посилання (`Controllers/AuthController.cs`)

```
[HttpPost("signup/start")]
public async Task<IActionResult> SignupStart([FromBody] SignupStartRequest req,
Cancellation token ct)
{
    var email = req.Email?.Trim();
    if (string.IsNullOrEmpty(email)) return BadRequest("Email is required.");
    if (await _db.Users.AnyAsync(u => u.Email == email, ct))
        return Conflict("An account with this email already exists.");

    // Інвалідуємо всі попередні живі токени – діє лише найсвіжіше посилання.
    await _db.SignupTokens
        .Where(t => t.Email == email && t.ConsumedAt == null)
        .ExecuteUpdateAsync(s => s.SetProperty(t => t.ConsumedAt, DateTime.UtcNow), ct);

    var rawToken = SignupTokens.Generate();
    _db.SignupTokens.Add(new SignupToken
    {
        Email = email!,
        TokenHash = SignupTokens.Hash(rawToken),
        ExpiresAt = DateTime.UtcNow.AddMinutes(SignupTokenLifetimeMinutes),
        OptOutMarketing = req.OptOutMarketing
    });
    await _db.SaveChangesAsync(ct);

    var baseUrl = _emailOptions.PublicBaseUrl.TrimEnd('/');
    var link = $"{baseUrl}/signup/complete?token={Uri.EscapeDataString(rawToken)}";
    await _emailSender.SendAsync(email!, "You're almost there!", BuildSignupEmail(link),
ct);
    return NoContent();
}
```

Приклад 3 — Фіналізація реєстрації: перевірка токена, створення облікового запису й кореневого профілю (Controllers/AuthController.cs)

```
[HttpPost("signup/complete")]
public async Task<ActionResult<AuthResponse>> SignupComplete([FromBody]
SignupCompleteRequest req, CancellationToken ct)
{
    if (string.IsNullOrEmpty(req.Token)) return BadRequest("Token is required.");
    if (string.IsNullOrEmpty(req.Password) || req.Password!.Length < 6)
        return BadRequest("Password must be at least 6 characters.");

    var entity = await _db.SignupTokens.FirstOrDefaultAsync(t => t.TokenHash ==
SignupTokens.Hash(req.Token!), ct);
    if (entity is null) return BadRequest("Invalid sign-up link.");
    if (entity.ConsumedAt is not null) return BadRequest("This link has already been
used.");
    if (entity.ExpiresAt < DateTime.UtcNow) return BadRequest("This link has expired.");

    var (subscriptionType, subscriptionExpiration) = (req.Plan ??
"trial").Trim().ToLowerInvariant() switch
    {
        "unlimited" => ("Unlimited",
(DateOnly?)DateOnly.FromDateTime(DateTime.UtcNow.AddDays(30))),
        "comics"   => ("Comics",
(DateOnly?)DateOnly.FromDateTime(DateTime.UtcNow.AddDays(30))),
        "tv"       => ("TV",
(DateOnly?)DateOnly.FromDateTime(DateTime.UtcNow.AddDays(30))),
        _          => ("Trial",
(DateOnly?)DateOnly.FromDateTime(DateTime.UtcNow.AddDays(7))),
    };

    var user = new User
    {
        Email = entity.Email,
        Username = await GenerateUniqueUsername(entity.Email, ct),
        Password = _hasher.Hash(req.Password!),
        Role = "USER",
        SubscriptionType = subscriptionType,
        SubscriptionExpiration = subscriptionExpiration
    };
    _db.Users.Add(user);
    entity.ConsumedAt = DateTime.UtcNow;
    await _db.SaveChangesAsync(ct);

    EnsureRootProfile(user); // атомарне створення кореневого профілю
    await _db.SaveChangesAsync(ct);
    return Ok(BuildResponse(user));
}
```

Приклад 4 — Формування й підписання маркера сесії JWT алгоритмом HMAC SHA-256 (Services/JwtService.cs)

```
public string GenerateToken(User user)
{
    var role = string.IsNullOrEmpty(user.Role) ? "USER" :
user.Role!.ToUpperInvariant();
    var claims = new List<Claim>
    {
        new(JwtRegisteredClaimNames.Sub, user.Email),
        new(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString()),
        new(ClaimTypes.NameIdentifier, user.Id.ToString()),
        new(ClaimTypes.Name, user.Username),
        new(ClaimTypes.Email, user.Email),
        new(ClaimTypes.Role, role),
    };
};
```

```

var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_options.Secret));
var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);
var token = new JwtSecurityToken(
    issuer: _options.Issuer, audience: _options.Audience, claims: claims,
    notBefore: DateTime.UtcNow,
    expires: DateTime.UtcNow.AddMilliseconds(_options.ExpirationMs),
    signingCredentials: creds);
return new JwtSecurityTokenHandler().WriteToken(token);
}

```

Приклад 5 — Адаптивне BCrypt-згорання та перевірка паролів і PIN-кодів (Services/BCryptPasswordHasher.cs)

```

public class BCryptPasswordHasher : IPasswordHasher
{
    public string Hash(string password) => BCrypt.Net.BCrypt.HashPassword(password);

    public bool Verify(string password, string hash)
    {
        try { return BCrypt.Net.BCrypt.Verify(password, hash); }
        catch { return false; }
    }
}

```

Приклад 6 — Перевірка коду TOTP із допуском відхилення годинника (RFC 6238) (Security/Totp.cs)

```

public static bool Verify(string? secret, string? code, int window = 1)
{
    if (string.IsNullOrEmpty(secret) || string.IsNullOrEmpty(code)) return false;
    code = code.Trim();
    if (code.Length != Digits || !code.All(char.IsDigit)) return false;

    byte[] key;
    try { key = Base32Decode(secret); } catch { return false; }

    var counter = DateTimeOffset.UtcNow.ToUnixTimeSeconds() / PeriodSeconds;
    for (var offset = -window; offset <= window; offset++) // ±1 інтервал (±30 с)
        if (FixedTimeEquals(Compute(key, counter + offset), code)) return true;
    return false;
}

private static string Compute(byte[] key, long counter)
{
    var counterBytes = BitConverter.GetBytes(counter);
    if (BitConverter.IsLittleEndian) Array.Reverse(counterBytes);
    using var hmac = new HMACSHA1(key);
    var hash = hmac.ComputeHash(counterBytes);
    var off = hash[^1] & 0x0F; // динамічне зрізання
    var binary = ((hash[off] & 0x7F) << 24) | ((hash[off + 1] & 0xFF) << 16)
        | ((hash[off + 2] & 0xFF) << 8) | (hash[off + 3] & 0xFF);
    return (binary % (int)Math.Pow(10, Digits)).ToString().PadLeft(Digits, '0');
}

```

Приклад 7 — Обмін одноразового виклику 2FA на маркер сесії після перевірки коду (Controllers/AuthController.cs)

```

[HttpPost("2fa/login-verify")]
public async Task<ActionResult<AuthResponse>> TwoFactorLoginVerify(
    [FromBody] TwoFactorLoginVerifyRequest req, CancellationToken ct)
{
    if (string.IsNullOrEmpty(req.ChallengeToken)) return BadRequest("Challenge token is required.");
}

```

```

    if (string.IsNullOrEmpty(req.Code)) return BadRequest("Code is required.");

    var hash = SignupTokens.Hash(req.ChallengeToken!);
    var challenge = await _db.TwoFactorChallenges.FirstOrDefaultAsync(c => c.TokenHash ==
hash, ct);
    if (challenge is null || challenge.ConsumedAt is not null || challenge.ExpiresAt <
DateTime.UtcNow)
        return BadRequest("This verification step has expired. Please sign in again.");

    var user = await _db.Users.FirstOrDefaultAsync(u => u.Id == challenge.UserId, ct);
    if (user is null || string.IsNullOrEmpty(user.TwoFactorSecret)) return
BadRequest("Account not found.");
    if (!Totp.Verify(user.TwoFactorSecret, req.Code)) return BadRequest("Incorrect code. Try
again.");

    challenge.ConsumedAt = DateTime.UtcNow; // одноразовість виклику
    await _db.SaveChangesAsync(ct);
    return Ok(BuildResponse(user)); // лише тут видається маркер
ceciі
}

```

Приклад 8 — Встановлення та перевірка PIN-коду профілю з батьківським контролем (Controllers/ProfilesController.cs)

```

[HttpPut("{id:long}/pin")]
public async Task<IActionResult> SetPin(long id, [FromBody] SetPinRequest req,
Cancellation token ct)
{
    var user = await CurrentUser.GetAsync(User, _db, ct);
    if (user is null) return Unauthorized();
    var profile = await _db.Profiles.FirstOrDefaultAsync(p => p.Id == id && p.UserId ==
user.Id, ct);
    if (profile is null) return NotFound();
    if (string.IsNullOrEmpty(req.Pin) || !PinFormat.IsMatch(req.Pin!)) // рівно 4 цифри
        return BadRequest("PIN must be exactly 4 digits.");

    // Заміна наявного PIN потребує поточного — окрім кореневого профілю (батьківський
контроль).
    if (!string.IsNullOrEmpty(profile.PinHash) && !await CanBypassPinAsync(user, ct))
        if (string.IsNullOrEmpty(req.CurrentPin) || !_hasher.Verify(req.CurrentPin!,
profile.PinHash))
            return BadRequest("Current PIN does not match.");

    profile.PinHash = _hasher.Hash(req.Pin!);
    await _db.SaveChangesAsync(ct);
    return NoContent();
}

```

Приклад 9 — Створення Customer і SetupIntent платіжного шлюзу з обмеже-ним тайм-аутом (Services/StripeBillingService.cs)

```

public StripeBillingService(IOptions<StripeOptions> options, ILogger<StripeBillingService>
logger)
{
    _options = options.Value;
    if (IsConfigured)
    {
        // Тісний тайм-аут і один повтор: заблоковане з'єднання падає швидко, а не
«зависає».
        var httpClient = new HttpClient { Timeout = TimeSpan.FromSeconds(12) };
        _stripeClient = new StripeClient(_options.SecretKey,
            httpClient: new SystemNetHttpClient(httpClient, maxNetworkRetries: 1));
    }
}

```

```

public async Task<SetupIntentResult> CreateSetupIntentAsync(string email, CancellationToken
ct = default)
{
    var customer = await new CustomerService(_stripeClient)
        .CreateAsync(new CustomerCreateOptions { Email = email }, cancellationTokens: ct);
    var intent = await new SetupIntentService(_stripeClient).CreateAsync(
        new SetupIntentCreateOptions
        {
            Customer = customer.Id,
            PaymentMethodTypes = new List<string> { "card" },
            Usage = "off_session",
        }, cancellationTokens: ct);
    return new SetupIntentResult(intent.ClientSecret, customer.Id); // секрет картки на
сервер не потрапляє
}

```

Приклад 10 — Диференційоване зберігання стану та обробка виклику 2FA на клієнті (contexts/AuthContext.tsx)

```

// Відомості акаунта та маркер – довгостроково; активний профіль – лише на сесію.
const [token, setToken] = useState<string | null>(() => localStorage.getItem("auth_token"));
const [currentProfileId, setCurrentProfileId] = useState<number | null>(() => {
    const raw = sessionStorage.getItem(CURRENT_PROFILE_KEY);
    return raw ? Number(raw) : null;
});

// Перший фактор: якщо увімкнено 2FA – повертаємо виклик замість завершення сесії.
const applyFirstFactor = (response: AuthApiResponse): LoginOutcome => {
    if (response.twoFactorRequired && response.challengeToken) {
        return { twoFactorRequired: true, challengeToken: response.challengeToken };
    }
    handleAuth(response.token, response);
    return { twoFactorRequired: false };
};

```

Приклад 11 — Охоронні компоненти маршрутизації клієнта (App.tsx)

```

const RequireProfile = ({ children }: { children: React.ReactNode }) => {
    const { user, currentProfile, trialExpired, profiles, profilesLoading } = useAuth();
    if (!user) return <Navigate to="/auth" replace />;
    if (!currentProfile) {
        if (usingStoredProfile(profiles, profilesLoading)) return <ProfilesLoading />;
        return <Navigate to="/profiles" replace />; // не обрано профіль
    }
    if (trialExpired) return <Navigate to="/bazinga-unlimited?from=trial-expired" replace />;
    return <>{children}</>;
};

```

Приклад 12 — Підтвердження виклику 2FA одноразовим кодом на клієнті (lib/auth.ts)

```

/** Обмін одноразового виклику + коду TOTP на сесію. */
export const twoFactorLoginVerify = (challengeToken: string, code: string) =>
    apiFetch<SigninVerifyResponse>("/api/auth/2fa/login-verify", {
        method: "POST",
        body: JSON.stringify({ challengeToken, code }),
    });

```